

AKINOLA ADEJOKE

Full-stack & blockchain developer

akinolaa769@gmail.com

github.com/natureloved

www.linkedin.com/in/ak...

x.com/RastaDev_

Full-stack and protocol engineering across Bitcoin L2s, EVM, Starknet, Monad, Base and Solana. TypeScript end to end: smart contracts, backends, and the interfaces that sit on top of them.

Currently a first-year radiography student, and building alongside it. Both fields come down to the same thing: understand the system properly and refuse to guess.

109

automated tests

12

live demos

12

open-source projects

SELECTED WORK

DrawBound

drawbound-eight.vercel.app/vault

Borrow against your Bitcoin without ever handing it over.

TypeScript · Next.js · SatVM · Tachi signet · BIP-340 · HAT/RIP

Acadex

acadex-gamma.vercel.app

A university exam calendar that can't quietly go out of date.

Next.js 16 · React Server Components · TypeScript · Prisma 7 · Tailwind 4

PegWatch

pegwatch-neon.vercel.app

An agent that keeps watch over the hours when the price feeds fall asleep.

TypeScript · Base · Aerodrome · Chainlink · viem

Voz

voz-three.vercel.app

Send money by saying it out loud. The person receiving it never has to read anything.

TypeScript · Next.js · ElevenLabs · Claude · LI.FI · Solana

TipWall

tipwall.vercel.app

A support wall that pays the creator in full, in the currencies people actually hold.

Nimiq Pay · Ed25519 · TypeScript

Every project here exists because something was harder than it needed to be.

An exam calendar that took a department a week to build and could quietly go out of date. A withdrawal threshold that stranded small balances. A remittance flow that assumed the person receiving the money could read a wallet address. A support wall that took a cut of the money it was meant to pass through.

That is the whole instinct: find the friction, understand it properly, remove it, and hand someone a thing that just works. If a problem needs a spreadsheet and a process change rather than an app, that is the answer too.

ALSO SHIPPED

Proof of Rest

proof-of-rest.vercel.app

Lock a stake to start a work session. Stop on time and reclaim it; run over and lose part. The cool-down is enforced by the contract.

Solidity · Monad

Staxiq

staxiq.vercel.app

Where should your Bitcoin sit? A yield view for Bitcoin DeFi on Stacks, scored by an open rubric that admits when its numbers disagree with DefiLlama.

Clarity · Stacks

Expatship

[expatship.vercel.app](#)

Cross-border shipping for people moving country: instant duty-inclusive quote, generated customs invoice, live route map. It labels what is computed and what is sample data.

React · Vite · Supabase

Deadman Vault

[deadman-vault-eta.vercel.app](#)

A proof-of-life treasury. Check in on schedule and your funds stay put; stop and they move to the people you chose.

Stacks · Clarity

HashPilot

[hashpilot-taupe.vercel.app](#)

A mining terminal that reads chain state and gives a straight answer: claim now, or wait. Plus a three-move playbook for your risk profile.

Next.js · Avalanche

TonPilot

[tonpilot.vercel.app](#)

A Telegram bot where you describe what you want done with your wallet in plain language, and it does it. No dashboard, no dApp.

TypeScript · TON

StashFlow

[stashflow-two.vercel.app](#)

Goal-based savings where the goal is the point: name an amount, watch it fill up, earn yield the whole time.

Next.js 16 · LI.FI

WHAT I DO

Web apps people actually finish using

Interfaces, auth, roles, payments, notifications, audit trails. Shipped end to end, not a prototype that stops at the login screen.

Smart contracts and the rails around them

Bitcoin L2, EVM, Starknet and Canton. The contract, the indexer, the relayer, the dashboard: the boring parts that make it real.

AI that is accountable for what it does

Agents that act on-chain, explain themselves, and are built so a human can inspect the audit trail afterwards.

TECHNICAL SKILLS

Languages:	TypeScript, JavaScript, Python, Clarity, Solidity
Chains:	Stacks (Bitcoin L2), EVM, Starknet, Monad, Base, Solana, TON, Nimiq
Frontend:	Next.js, React, TypeScript, Tailwind CSS, Recharts
Backend & data:	Node.js, Python, Supabase, SQLite, REST APIs, DeFiLlama
Web3 tooling:	Clarinet, Foundry, wagmi, Stacks Connect, WalletConnect, LI.FI, SIP-009
AI:	Claude-powered intent parsing, copilots, auditable agents

EDUCATION

Radiography & radiation science

First year, currently enrolled

HOW I WORK

- Find the real problem — Most briefs describe a symptom. I go looking for the actual failure, who it hurts, how often, and what it costs them today.
- Decide what must be true — Before writing code I write down the rules the system cannot break, and what the user sees at each step. If I can't state it, I haven't understood it yet.
- Build it properly — Typed end to end, tests around the risky parts, honest about which data is real and which is illustrative.
- Show the evidence — A live URL, a test run, a screenshot. Anyone can claim to have built something; I'd rather you check than take my word.

Every claim here links to something you can check: the repositories are public and the deployments are live.